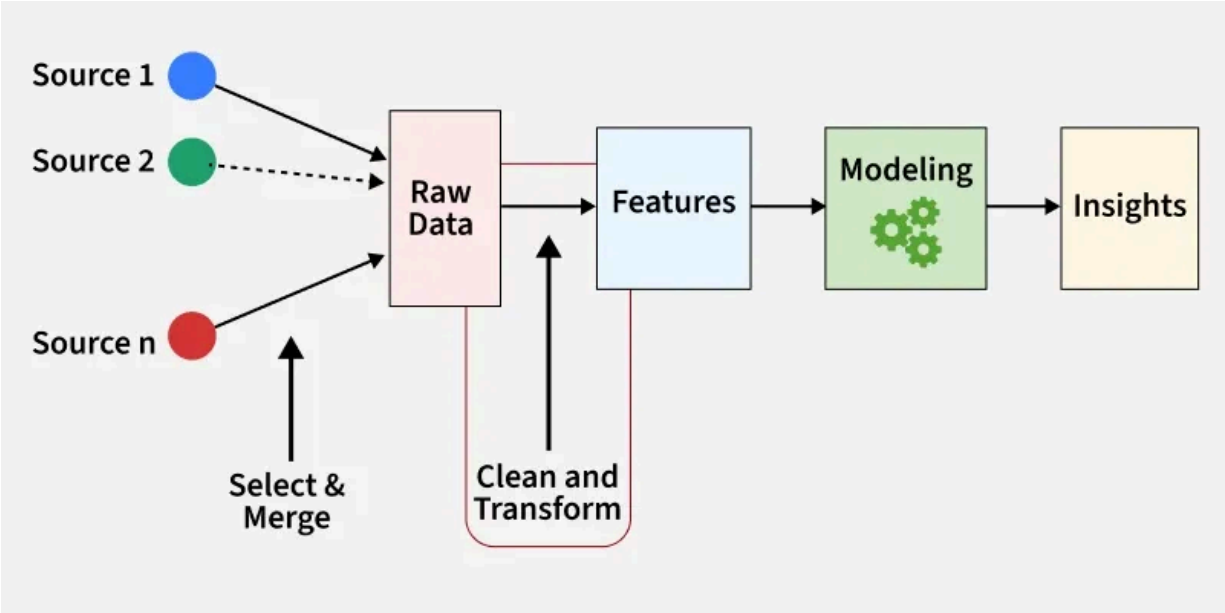
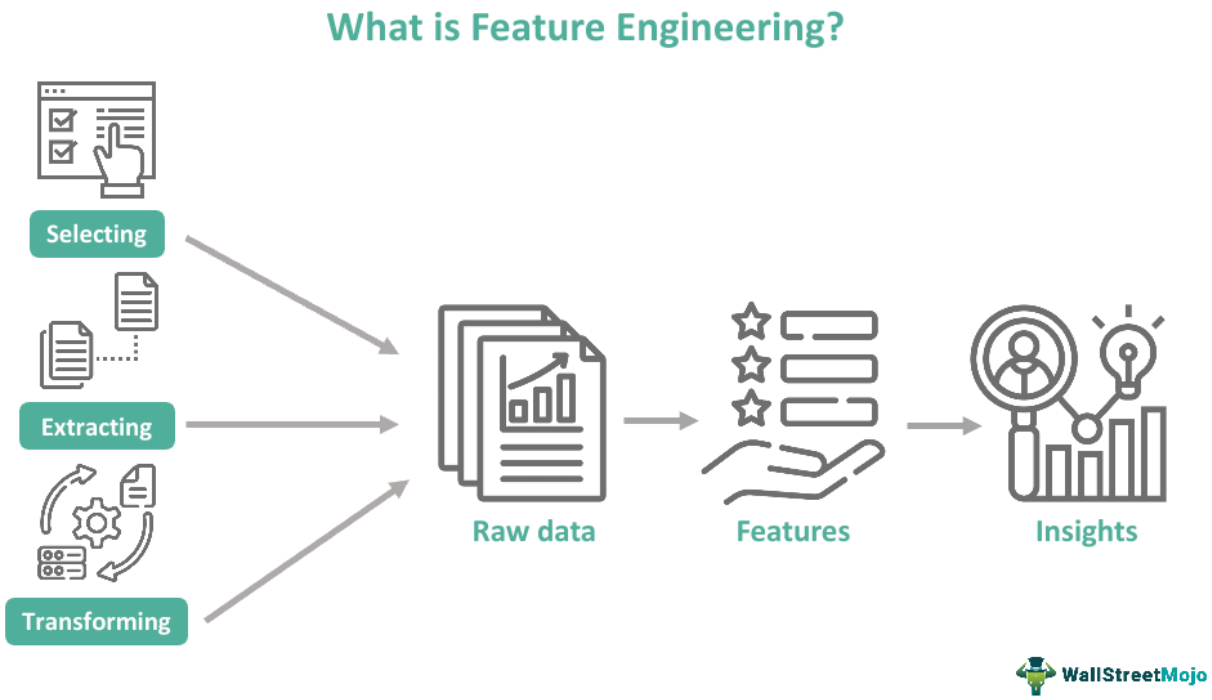
MODULE 3 — Feature Engineering: Representation as Power

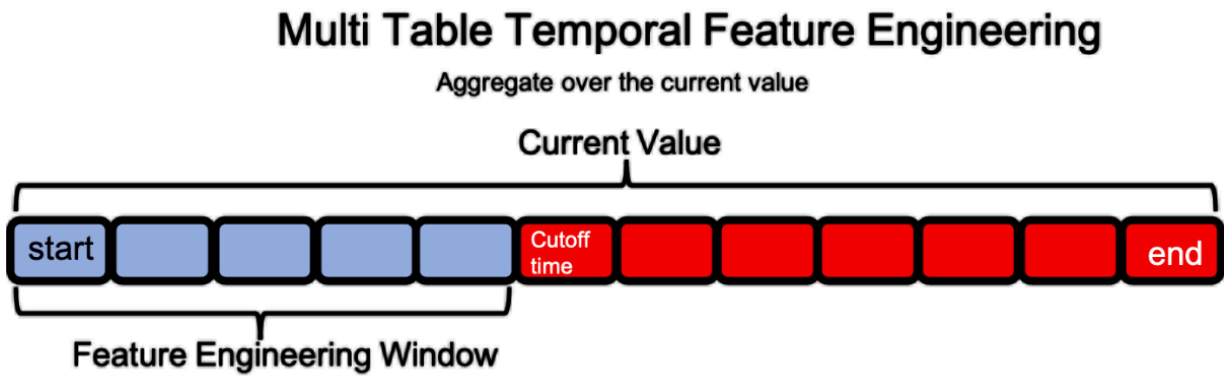
How the World Is Shown to the Model



https://www.geeksforgeeks.org/machine-learning/what-is-feature-engineering/?utm_source=chatgpt.com



https://www.wallstreetmojo.com/feature-engineering/?utm_source=chatgpt.com
https://www.researchgate.net/figure/The-process-of-leakage-feature-extraction_fig2_392831505



https://featuretools.alteryx.com/en/stable/guides/time_series.html?utm_source=chatgpt.com

Why This Module Matters

Models do **not** discover truth. They discover patterns *in the representation you give them*.
Feature engineering determines:

- what relationships are visible
- what invariances are preserved

- what behaviors are amplified or hidden
- what shortcuts the model can exploit

Once features exist, **entire classes of decisions are already locked in**.

Anchor Back to Our Decision

We are still deciding:
Which customer cohorts should receive churn interventions under budget and operational constraints, with uncertainty about whether recent behavioral changes reflect true churn risk or temporary noise.
Features are how that uncertainty is **encoded**.

Core Reframe (Memorize This)

Feature engineering is not column creation — it is choosing what the model is allowed to notice.
If the model "discovers" something you didn't intend, it's usually because you showed it that relationship.

Takeaways

- Feature engineering operates on **already-shaped data,** so the space of viable engineering strategies must respect *cleaned distributions, preserved signals, and accepted biases*
  - Any feature ideas outside of these constraints are not only undesirable, but are **illegal**
  - *"You can't engineer features that rely on uncertainty you already removed."*
- This fits the theme of approaching data cleansing, in that there's no "global approach" to building new features from existing data.
  - What features are created at this stage are directly dictated by what data cleansing strategy was accepted/approved.
  - **Key:** With every process step completed in the data science pipeline the breadth of the decision tree narrows. **-> specifically in a decision-chained data science pipeline**
  - **Due to:**
    - Irreversible commitments made earlier
    - Uncertainty being collapsed into structure step by step

A Very Useful Analogy (Worth Keeping)

Think of the pipeline like:

🧊 Freezing a Liquid

- Ingestion: liquid, high entropy
- Cleaning: thickening
- Features: semi-solid
- Models: rigid shape
- Production: frozen block

You can't reshape ice without melting it again — which means going back upstream.

AI's Feature Engineering Questions

Before proposing a single feature, AI implicitly asks:

1. **What relationships must the model see to support this decision?**
2. **What invariances should be preserved?** (What *shouldn't* matter?)
3. **What leakage risks exist?**
4. **What behavior could the model exploit instead of learning intent?**

If you can't answer these, feature work is premature.

Representation Changes Reality

Consider these equivalent raw facts:

- "Usage dropped by 40%"
- "Usage percentile dropped from 70th → 20th"
- "User crossed inactivity threshold"

Each:

- encodes a different worldview
- implies a different intervention policy
- creates different failure modes

The model doesn't choose — **you do**.

Leakage Is a Representation Problem (Not a Modeling One)

Leakage is not about what model you choose — it's about what version of reality you show the model.

Once a leaked feature exists:

- any model will use it
- better models exploit it *faster*
- evaluation metrics become meaningless

Leakage often looks like:

- features derived *after* the decision point
- labels sneaking back in via aggregates

- proxy variables that encode outcomes

But conceptually, leakage is:

Giving the model information it would not have at decision time.

Once leaked, no evaluation can save you.

Appendix

- Under Module 3, see *What Is Leakage & When Does It Occur?*

A: Data leakage occurs when the system is given information that would not be available at the moment the decision is made.

- **Decision** = the real-world moment when an action must be taken with the information available *at that time*.
- In other words, the system is cheating without realizing it
- In leakage settings, it's possible for
  - The data to be *technically correct*
  - The feature to be *highly predictive*
  - The evaluation to *look excellent*
- ... and the system is still invalid.

A: Most common leakage situations

- **Label leakage** = giving the model information that exists *because* the outcome already happened.
- **Temporal Leakage (Most Common)**
  - Features computed using future data -> aggregations that include post-decision behavior
  - Examples: *total usage over the next 30 days*, *number of support tickets in the quarter* (when decision happens at mid-quarter)
- **Label Leakage (Extremely Dangerous)**
  - Features that directly or indirectly encode the outcome
  - Direct Label Leakage - response variable accidentally appears in the input features
  - Indirect / Proxy Label Leakage - problematic feature isn't the label (response variable) itself, it's additional features storing outcomes of an event that already happened
    - Example: after a customer churn, internal systems assign
      - *a closure code, a reason flag, an account state* -> e.g., still giving the model the answer during training & testing, just not through the response variable.
- **Proxy Leakage (Very Subtle)**
  - Features that act as stand-ins for the label
  - Examples: "*Customer contacted by retention team*" predicting churn, "*manual review required*" *flag, escalation status*

    - Occurs through business processes and/or human interventions handling real world events
        - Similar issue to label leakage, except for indirect (e.g., non of the examples say whether a given record *has* been churned, but the feature examples indicate special-case events leading up to a churn event)
- **Aggregation Leakage**
    - Group-level statistics that include future or label-dependent data
    - Examples: *Average churn rate by cohort* computed using full dataset, normalizing features using global statistics (e.g., everything available to date) instead of historic ones
- **Feedback Loop Leakage (Advanced, Real Systems)**
    - Model outputs influencing future inputs
    - Examples: *customers flagged as high risk receive intervention, intervention changes behavior, behavior feeds back into model as "evidence"*
    - Occurs when there's no separation between an *observation* and an *intervention* or when production systems don't have proper guardrails

Takeaways

- Whether through requesting a data dictionary, other forms of requirements gathering, or stakeholder engagement, notes need to be made about what event-based features are created through business logic and/or human intervention. -> AI might not have access to this context!
- Basically, make sure to exclude

Temporal & Aggregation Choices Matter

Temporal feature design is how you tell the system *what kind of change matters*.

Churn signals are inherently temporal:

- **Short-term volatility** → rapid, noisy changes (day-to-day usage swings)
- **Medium-term trends** → sustained directional movement (weeks of decline)
- **Long-term decay** → slow erosion over long horizons (months of disengagement)

Feature windows:

- smooth or amplify noise
- shift detection timing
- change false-positive vs false-negative balance

These choices **encode bias**, just like cleaning did. -> this is because AI doesn't "see" these unless you **encode time intentionally**.

Appendix

- Under Module 3, see *How Is Feature Engineering of Temporal Signals Approached When Designing Pipelines that Prioritize Type I Error vs Type II*

Prompt Engineering Pattern (Critical)

When involving AI in feature design, use:

"Given this decision goal, propose features and explicitly label which ones are risky, leaky, or assumption-heavy."

Never accept a feature list without risk annotations.

🧪 Module 3 Mini-Exercise — Feature Risk Awareness

We'll stay conceptual — no code.

Step 1 — Propose Candidate Features

From our churn scenario, imagine **3 plausible features** derived from:

- usage logs
- billing history
- support interactions

You don't need formulas — descriptions are enough.

Step 2 — For *Each* Feature, Answer:

1. What relationship does this feature make visible?
2. What assumption does it encode?
3. What risk does it introduce?(leakage, bias, shortcut learning, false confidence)

Rules (Important)

❌ No models
❌ No metrics
❌ No "industry best practice"
✅ Explicit assumptions
✅ Risk labeling for *each* feature

Why This Module Is Subtle (And Dangerous)

Bad features don't look wrong.They look **useful**.

Models trained on bad representations:

- perform well
- fail silently
- cannot be defended

AI will happily exploit whatever you give it — unless you constrain it.

When You're Ready

Reply with:

- **Feature 1** + 3 answers
- **Feature 2** + 3 answers
- **Feature 3** + 3 answers

I'll review them by:

- surfacing hidden leakage
- identifying representational bias
- showing how each feature shapes downstream decisions

Then we'll move to **Module 4 — Modeling: Constraint Satisfaction, Not Popularity**.
Take your time — this module separates *good* systems from *dangerous* ones.